

Tito's Terminal 0.8

Terminal para Telnet, y SSH

MANUAL DE USUARIO

Por: Tito Hinostroza
Modificado: 04/02/2025

1 NOTAS

1.1 Notas sobre la versión 0.8

Esta versión da el salto a una interfaz basada en pestañas, en lugar de la ventana única de las versiones anteriores.

Se desecha el concepto de “script” o “Panel de comando” y ahora se maneja un tipo de documento llamado “sesión” que incluye siempre a un Panel de comandos y a una pantalla de terminal.

También se ha cambiado las opciones de configuración, como consecuencia de manejar ahora “sesiones” que incluyen muchas de las opciones de configuración, que antes eran configuraciones globales.

Quedan todavía varias opciones que actualizar para que se adapten mejor al cambio, pero se espera que se corrijan en las versiones siguientes.

1.2 Notas sobre la versión 0.7

Esta versión actualiza algunas librerías a versiones más recientes y se agregan funciones nuevas al intérprete de macros.

Se incluye también algunas mejoras en cuanto a la apariencia de la pantalla principal.

1.3 Notas sobre la versión 0.6

La versión 0.6 solo actualiza algunas librerías a las versiones más recientes.

1.4 Notas sobre Tito's Terminal

Tito's Terminal, es un programa libre (Ver Licencia).

Desarrollé Tito's Terminal como una necesidad para desarrollar mi trabajo diario. No está pensado para ser un cliente completo que cumpla con todas las especificaciones del protocolo Telnet o SSH.

Solo he implementado soporte a las principales secuencias de control del protocolo. No he implementado el reconocimiento del color o algunos caracteres de control. Sin embargo, el programa tiene una característica interna que colorea las palabras que van llegando por el puerto.

Está diseñado para trabajar principalmente con intérpretes de comando Linux tipo "bash" o "korn". Solo implementa un terminal en modo VT100 de 80 columnas por 25 filas, pero el ancho real de la pantalla puede llegar a varios cientos de caracteres.

A pesar de su simplicidad, ese programa soporta automatización mediante un lenguaje propio que es muy parecido al lenguaje de programación del programa TeraTerm. Adicionalmente, incluye un editor con resaltado de sintaxis y ayuda contextual muy útil.

Va mi agradecimiento a comunidad de Lazarus, por su apoyo y respuesta a las consultas desde el foro.

1.5 ¿Por qué un cliente más?

Habiendo ya muchos clientes Telnet o SSH, la pregunta sería por qué crear uno más. Pues la respuesta es también común: Porque ninguno tenía las características que estaba buscando en un cliente de Telnet (en el año 2010). ¿Cuáles eran?

- Que incluyera un editor de texto independiente, para escribir los comandos. Este editor de texto debería ser independiente del “prompt” del terminal, que tiene opciones limitadas de edición. Además, el editor debería tener opción, deshacer, resaltado de sintaxis, menú contextual, etc. Y debería permitir enviar todo un script completo al shell.
- El editor de texto independiente debería poder mostrarse siempre visible al lado del terminal (o abajo) y poder separarse como una ventana independiente.
- Debe poder cambiarse rápidamente con “Ctrl+Tab”, entre la ventana del editor y la ventana del terminal para poder ingresar los comandos a través del editor o directamente al terminal.
- El terminal (así como el editor independiente) debería tener resaltado de sintaxis en colores para el texto mostrado. Este coloreado del texto debería funcionar aún cuando el servidor remoto no envíe códigos de color (De hecho, no es necesario implementar el reconocimiento del color en el cliente Telnet). Debería también mecanismos para pintar el “prompt” mediante algún medio que le permita identificarlo en todo el flujo de texto recibido.
- La pantalla del terminal debería tener opciones de exploración, como si se tratara de editor normal. Debería poder soportar las opciones de búsqueda, seleccionar, copiar y pegar texto en el “prompt”, usando el teclado o el Mouse. Se debería poder mantener la ventana fija aunque lleguen caracteres al terminal.
- El terminal debería poder redimensionarse libremente, e independientemente del tamaño “virtual” del terminal Telnet. Para ellos debería manejar barras de desplazamiento.
- El terminal debería ser rápido, sin retrasos en la visualización, ni parpadeos. Además, debe permitir el registro del texto recibido.
- El cliente Telnet debería soportar automatización, a través de un lenguaje sencillo de macros, con un editor avanzado, con resaltado de sintaxis y ayuda contextual.
- Que sea portable. Sin necesidad de instalación.

Estos fueron los principales requerimientos que buscaba en un cliente de Telnet. Si existía uno que cumpliera todos estos requerimientos en el 2010, entonces mi trabajo ha sido en vano. Si no, entonces ha valido la pena.

1.6 Instalación

El programa se diseñó para no requerir instalación. Solo basta con copiar y descomprimir la carpeta que contiene los archivos del programa. El proyecto se encuentra alojado en <https://github.com/t-edson/Tito-s-Terminal>

El ejecutable es "TitoTerm.exe". Bastará con ejecutar este archivo para iniciar el programa.

Para la correcta ejecución del programa, se requiere que existan las siguientes carpetas:

\languages -> Aquí deben estar los archivos de sintaxis de los lenguajes soportados por la aplicación, cuando aplica el resaltado de sintaxis.

\sessions -> Aquí se guardan siempre, las sesiones de trabajo.

\macros -> Aquí se guardan por defecto, las macros creadas.

\temp -> Aquí se guardan los archivos temporales.

Las carpetas \sesiones y \macros, pueden estar vacías, pero si no se tienen archivos en la carpeta "languages", entonces no se podrá trabajar con las opciones de resaltado de sintaxis.

También es necesario tener el archivo "plink.exe", que es un cliente para conexiones telnet y SSH.

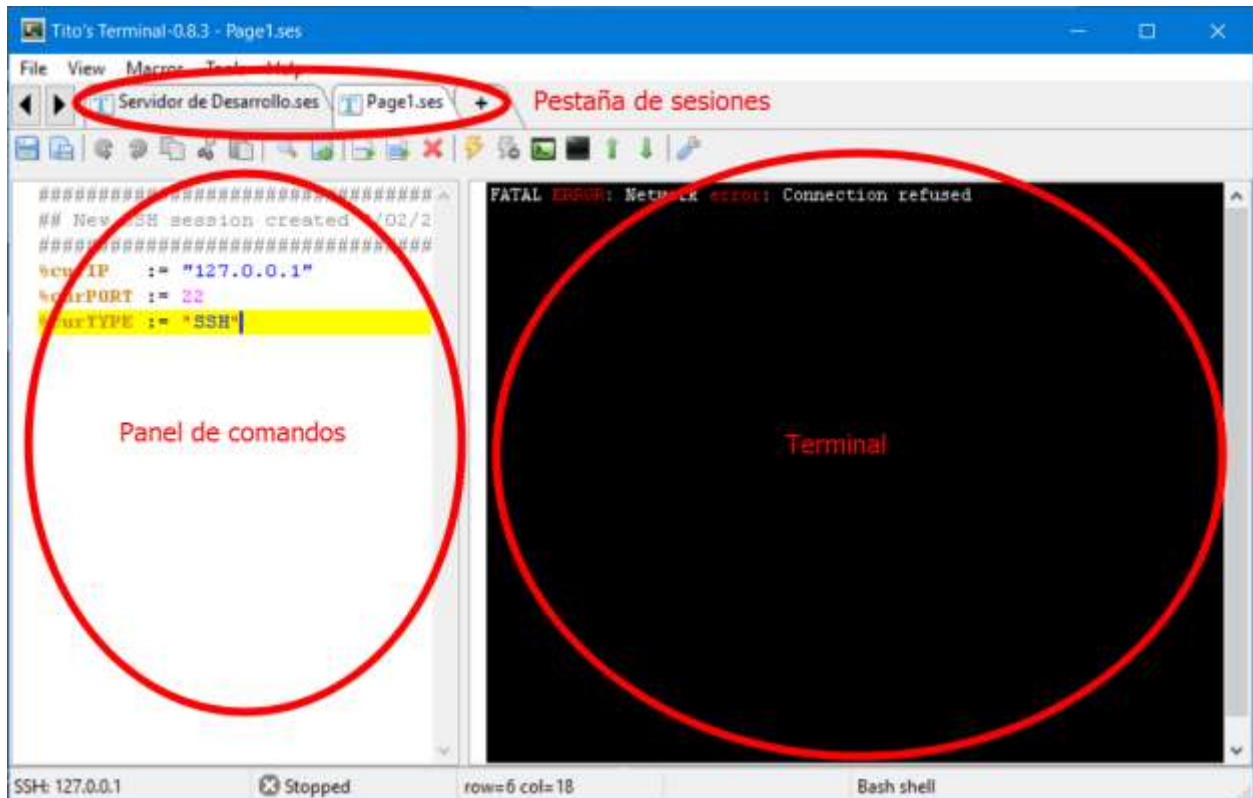
2 TABLA DE CONTENIDO

1	NOTAS.....	2
1.1	Notas sobre la versión 0.8.....	2
1.2	Notas sobre la versión 0.7.....	2
1.3	Notas sobre la versión 0.6.....	2
1.4	Notas sobre Tito's Terminal	3
1.5	¿Por qué un cliente más?	4
1.6	Instalación	5
2	TABLA DE CONTENIDO	6
3	FUNCIONAMIENTO.....	7
3.1	Interfaz	7
3.2	Sesiones.....	7
3.3	Panel de Comandos.....	11
3.4	Envío de comandos	14
3.5	El Terminal.....	17
3.6	Pantalla del Terminal.....	21
3.6.1	Coordenadas de un Terminal VT100 DE 80 * 25.....	22
3.6.2	Relación entre las 3 áreas	23
3.6.3	Manejo del Cursor	25
3.8	Archivos de texto.....	26
4	HERRAMIENTAS.....	27
4.1	Detección del Prompt.....	27
4.2	Explorador Remoto	29
4.3	Editor Remoto	31
4.4	Editor de macros	34
5	AUTOMATIZACIÓN	35
5.1	Variables y Tipos de datos.....	35
5.2	Estructura del lenguaje	36
5.3	Referencia a Instrucciones	37
5.4	Expresiones	39
5.5	Variables Predefinidas.....	39
5.6	Conexiones	40
5.7	Ejecución desde el Panel de Comandos.....	40

3 FUNCIONAMIENTO

3.1 Interfaz

La interfaz principal se compone de tres áreas principales:



El aplicativo puede manejar múltiples sesiones. Cada sesión se representa como una pestaña en la parte superior.

Cada sesión incluye un Panel de comandos y un Terminal.

Para moverse entre sesiones, se puede usar las combinaciones:

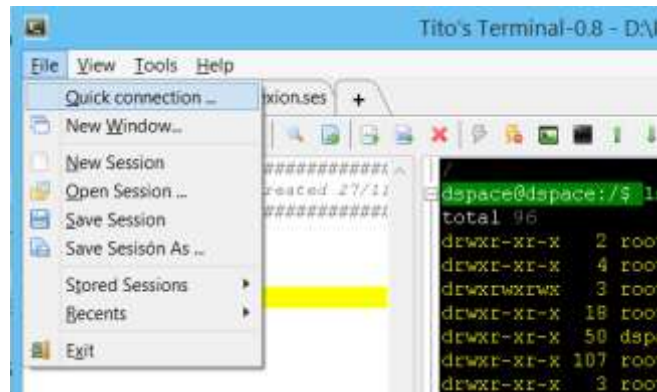
- <Ctrl>+<Tab> -> Se mueve a la siguiente sesión.
- <Shift>+<Ctrl>+<Tab> -> Se mueve a la sesión anterior.

3.2 Sesiones

Tito's Terminal maneja las conexiones a través de documentos llamados sesiones.

Una sesión es un conjunto de parámetros (de conexión, de contenido y de apariencia) que se pueden guardar como un solo archivo.

Las sesiones se administran desde el menú principal ">File":



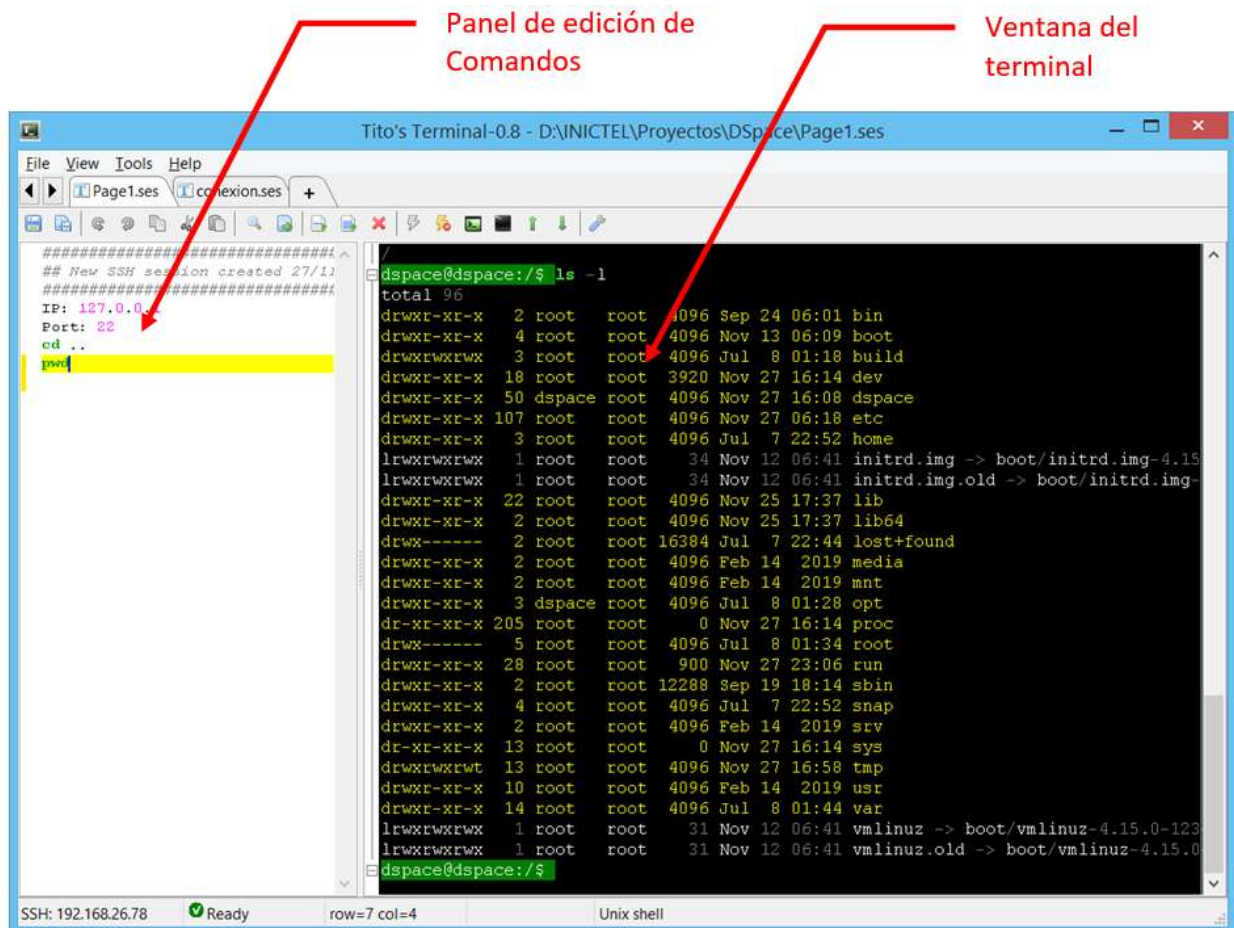
Se pueden abrir más de una sesión a la vez. Al crear una nueva sesión, se piden los parámetros de conexión iniciales:



Aquí se puede definir el tipo de conexión y los parámetros de la conexión. También se pueden definir parámetros adicionales, como la apariencia de la ventana o el comportamiento del terminal.

Todas estas configuraciones se guardan en la sesión y se pueden guardar también en disco.

Una sesión abierta presenta dos paneles principales:



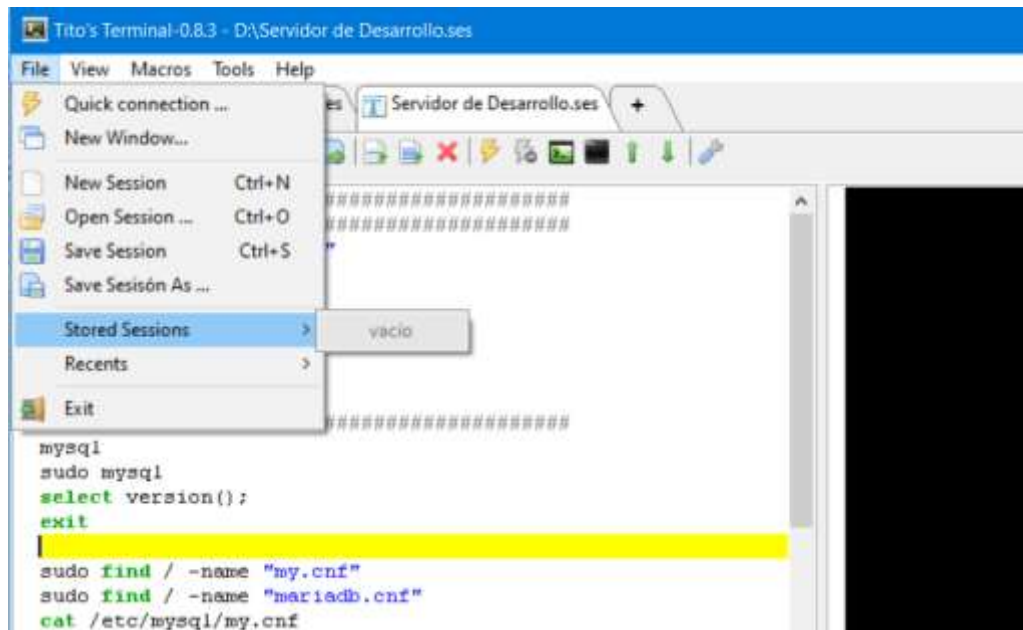
El panel de comandos puede mostrarse u ocultarse mediante el menú de configuraciones. También pueden cambiarse la apariencia y la distribución de los paneles.

El panel de comandos sirve para escribir los comandos que se van a lanzar en el terminal. La pantalla del terminal es de solo lectura.

Para pasar de un panel a otro se pueden usar los atajos:

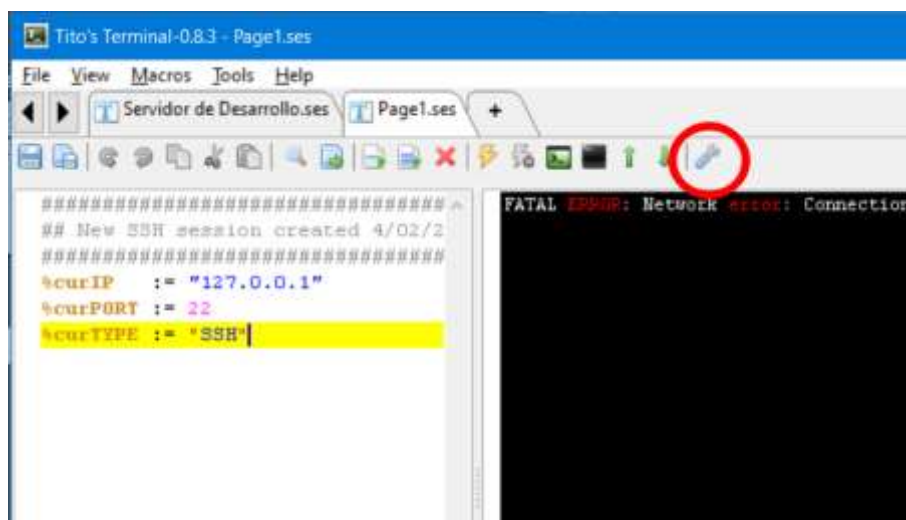
- <Ctrl>+1 -> Selecciona el panel de comandos.
- <Ctrl>+2 -> Selecciona al terminal.

Las sesiones pueden guardarse en cualquier parte del disco, pero las que se guarden en la carpeta /sessions serán accesibles directamente desde el menú principal.

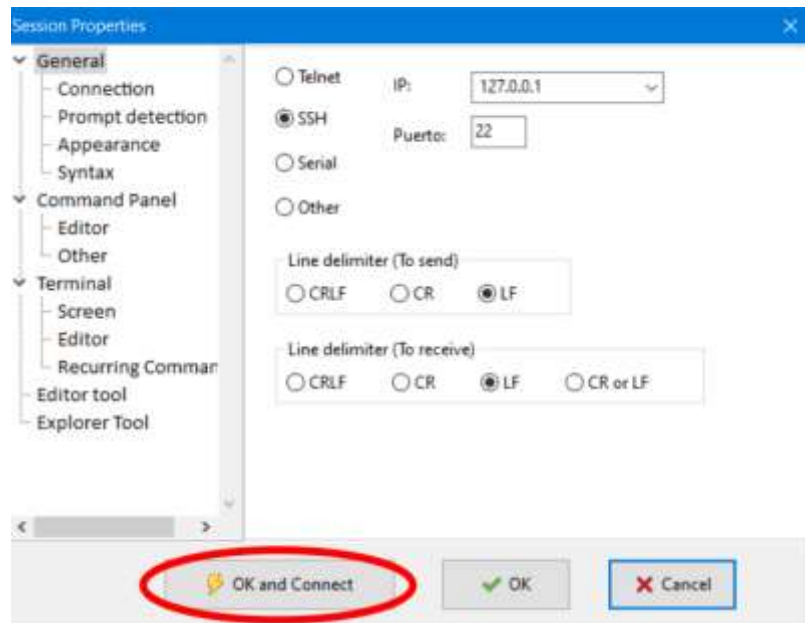


PROPIEDADES DE UNA SESIÓN

Se pueden acceder a las propiedades de una sesión desde el botón de configuración en la parte superior de la sesión:

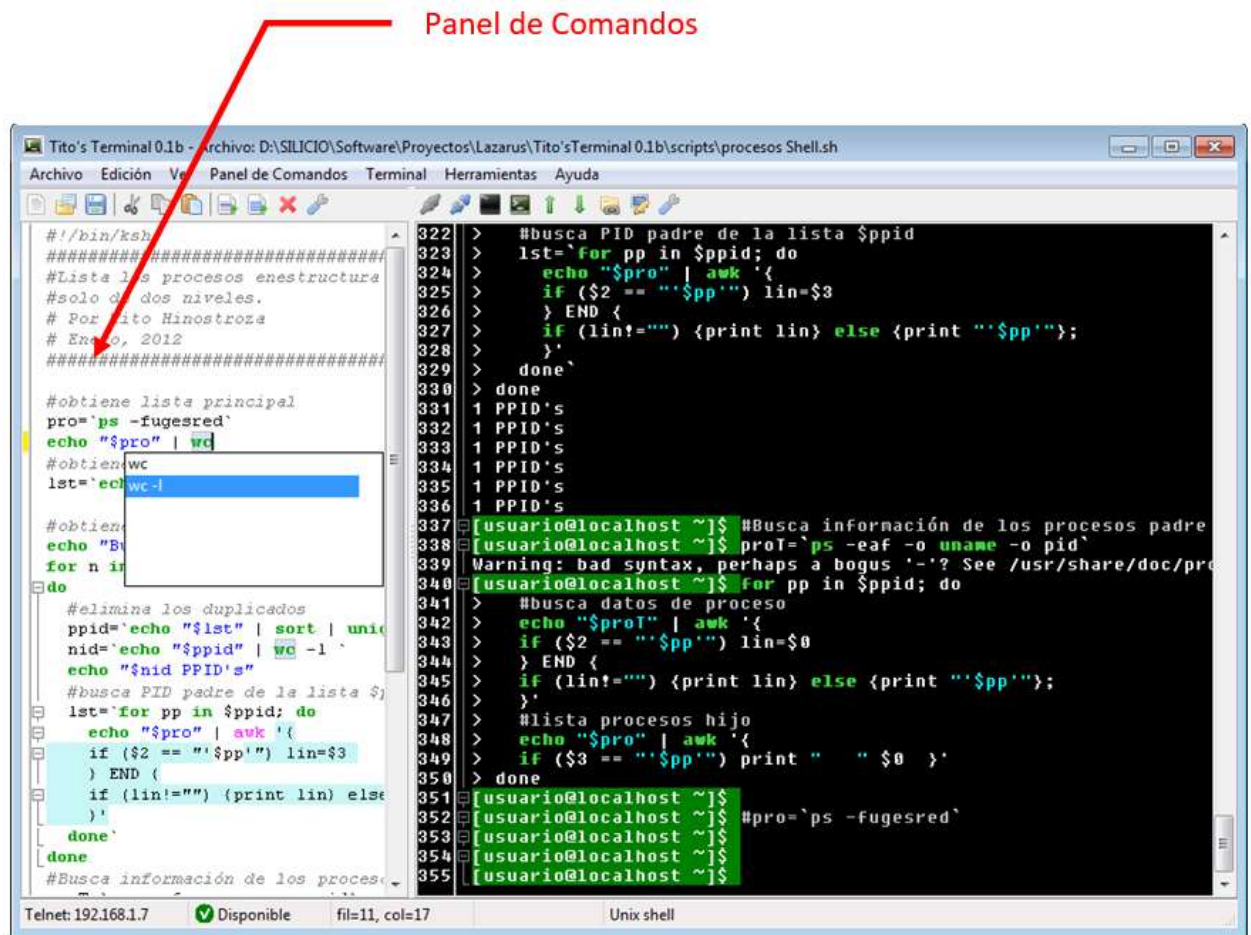


Después de cambiar la configuración de la sesión, se puede también iniciar la conexión de la sesión.



3.3 Panel de Comandos

El panel de comandos, de una sesión, es el panel lateral izquierdo, que permite escribir los comandos, antes de enviarlos al terminal. Si no se encuentra visible, se puede activar desde la ventana de configuración o con la tecla F2.



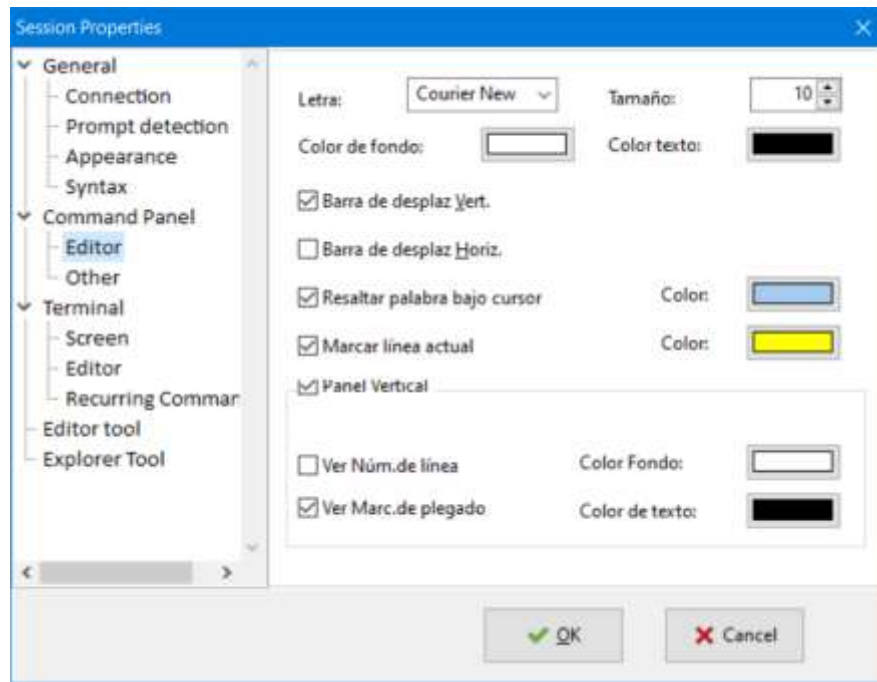
El Panel de comandos funciona como un editor de texto, con soporte para varios lenguajes (incluye resaltado de sintaxis, plegado de código y autocompletado de código).

Los comandos que se desean enviar al terminal deben escribirse siempre en el Panel de Comandos. No se pueden enviar comandos o teclas directamente desde el terminal

La función principal del Panel de comandos es poder enviar su contenido al terminal, sin tener que escribir el texto directamente en la ventana del terminal.

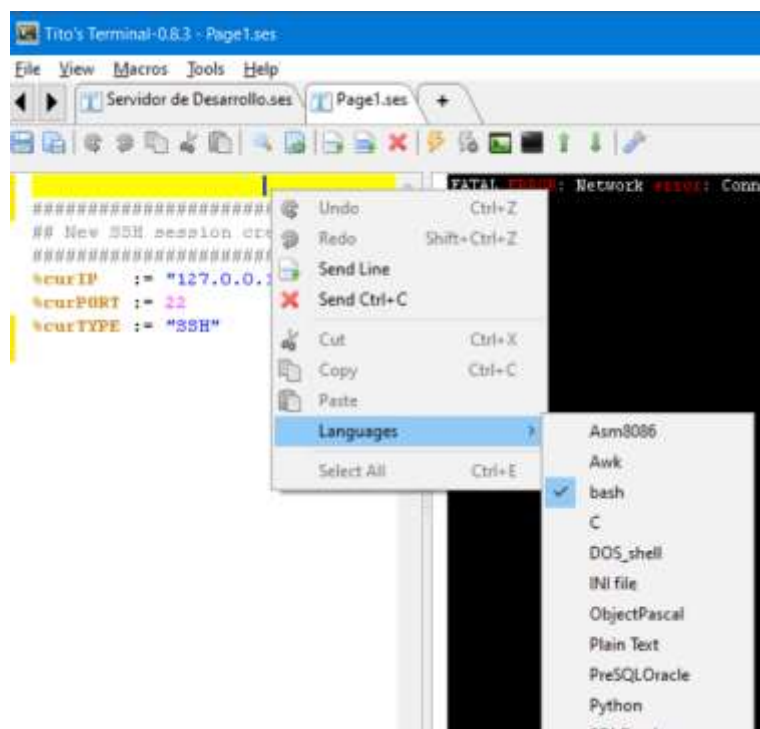
CONFIGURACIÓN DEL EDITOR

El editor del panel de comandos se puede configurar usando la ventana de configuración de la sesión:



Aquí se pueden configurar las características generales como el color de texto o del fondo, pero también se puede configurar para que el editor reconozca diversos lenguajes de programación y así activar el coloreado de sintaxis y el completado de código.

El lenguaje de trabajo del editor se elige desde el menú contextual que se abre en el Panel de Comandos:



El lenguaje configurado aparecerá en la barra de estado:



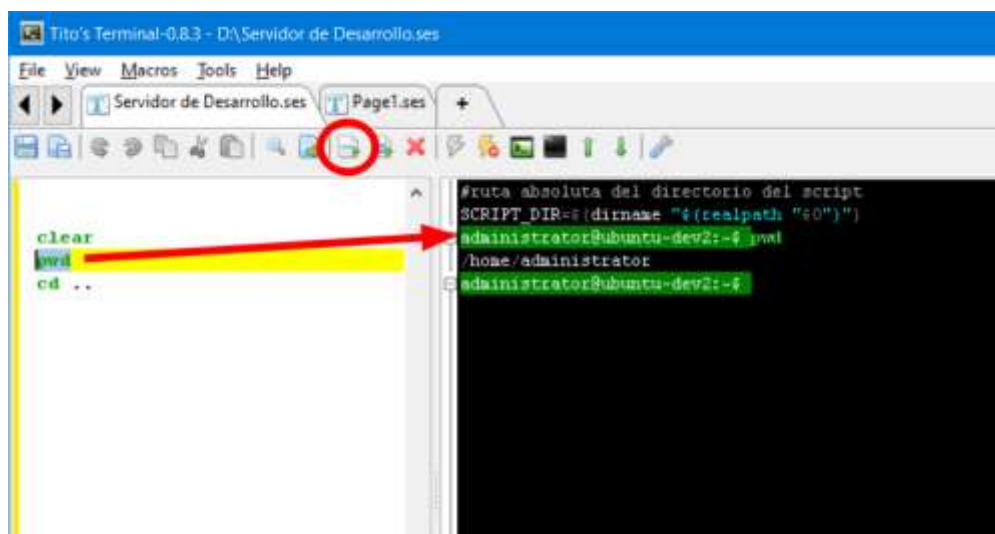
```
sudo apt purge "mariadb"
#Remover los paquetes asociados
sudo apt autoremove
#Ver el estado nuevamente
sudo systemctl status mariadb
#Debe salir "Unit mariadb.servic
```

SSH: 10.100.98.62 Ready row=1 col=1 Bash shell

3.4 Envío de comandos

Para el envío de comandos al terminal, se debe tener una sesión activa.

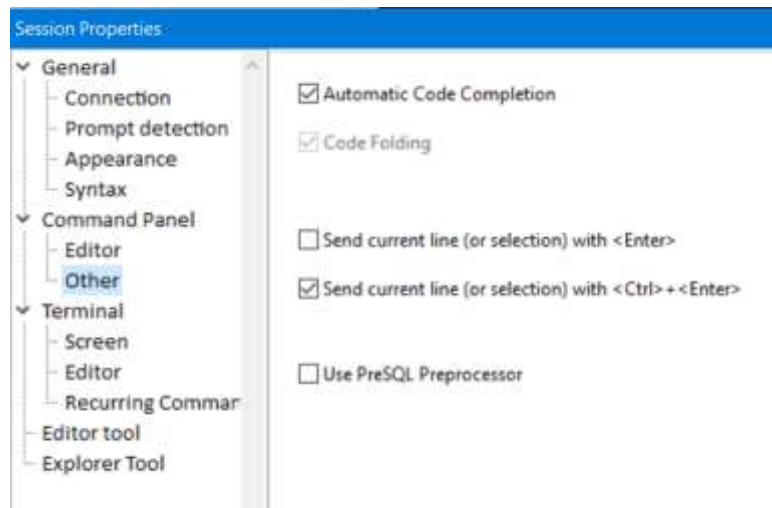
En el modo de trabajo normal, se van escribiendo las líneas y estas se van enviando con el botón de envío o con una combinación de teclas:



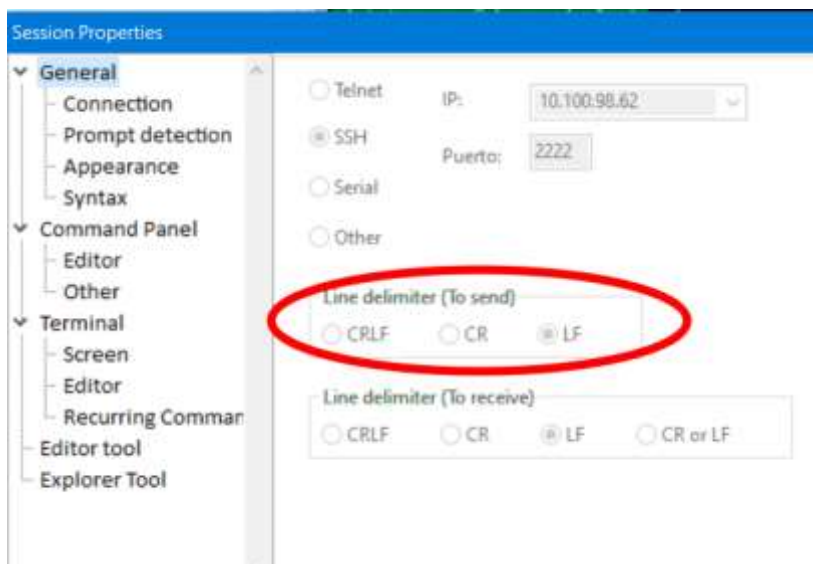
El editor de comandos permite, además, enviar el texto al terminal de las siguientes formas:

1. Enviar la línea actual o el texto seleccionado.
2. Enviar todo el texto del editor o el texto seleccionado.

La línea actual se envía usando las teclas <Enter> y/o <Ctrl>+<Enter>, dependiendo de cómo se haya hecho la configuración de la sesión:



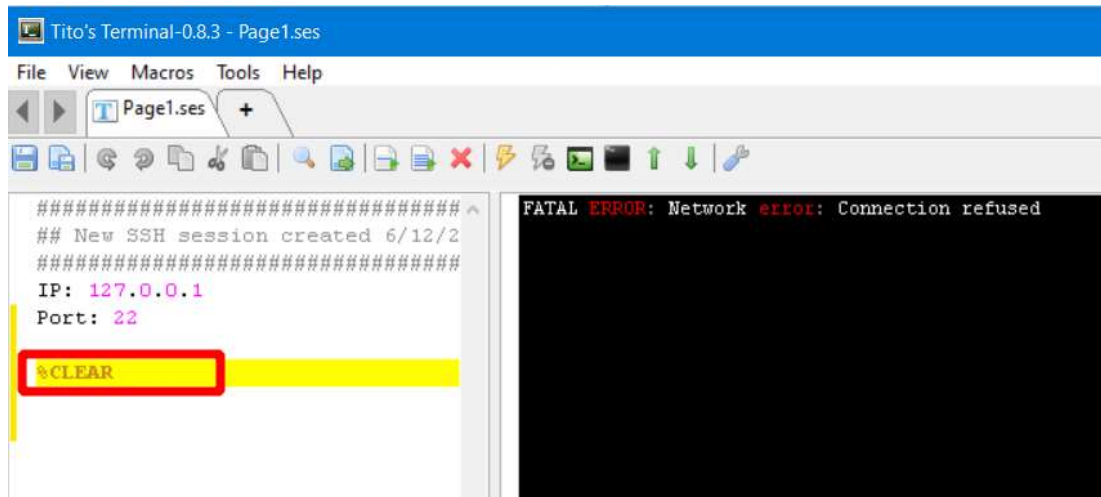
Al enviar una línea, esta se envía con un carácter de salto de línea al final, al terminal. Para configurar qué tipo de salto de línea se enviará, se puede acceder a la ventana de configuración:



La configuración de la conexión debe realizarse cuando la sesión está desconectada. De otra forma, no se permitirá cambiar los parámetros principales.

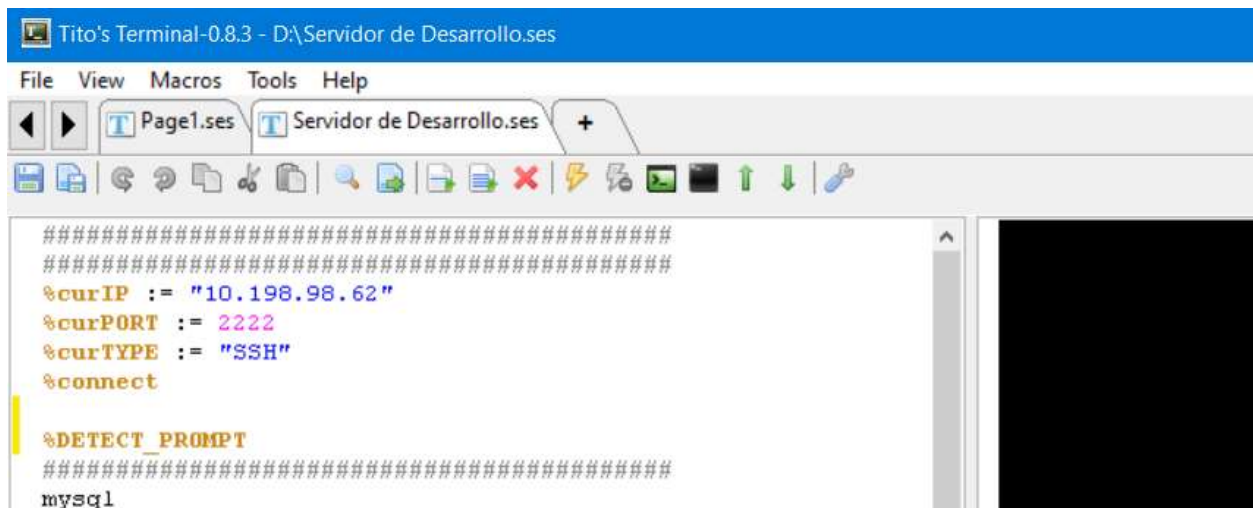
EJECUTANDO COMANDO DE MACROS

Adicionalmente a las líneas de texto que se pueden enviar directamente con <Enter> o <Ctrl>+<Enter>, es posible enviar también comandos del lenguaje de macros, como por ejemplo el comando para limpiar el terminal "CLEAR":

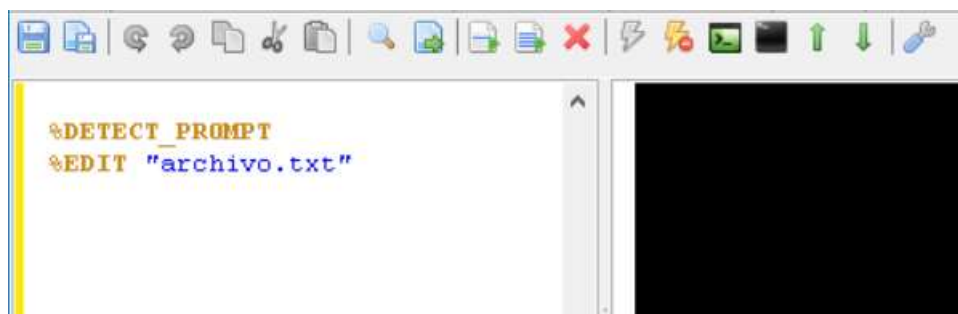


Los comandos de macros deben ser precedidos del carácter “%” para que se reconozcan como líneas especiales y no se envíen literalmente al terminal sino que se ejecuten como macros.

El siguiente ejemplo muestra los comandos que se necesitan para iniciar una conexión:



El siguiente comando permite editar un archivo de texto usando el editor remoto:



Para más información sobre los comandos existentes, revisar la sección de Automatización, más adelante en este documento.

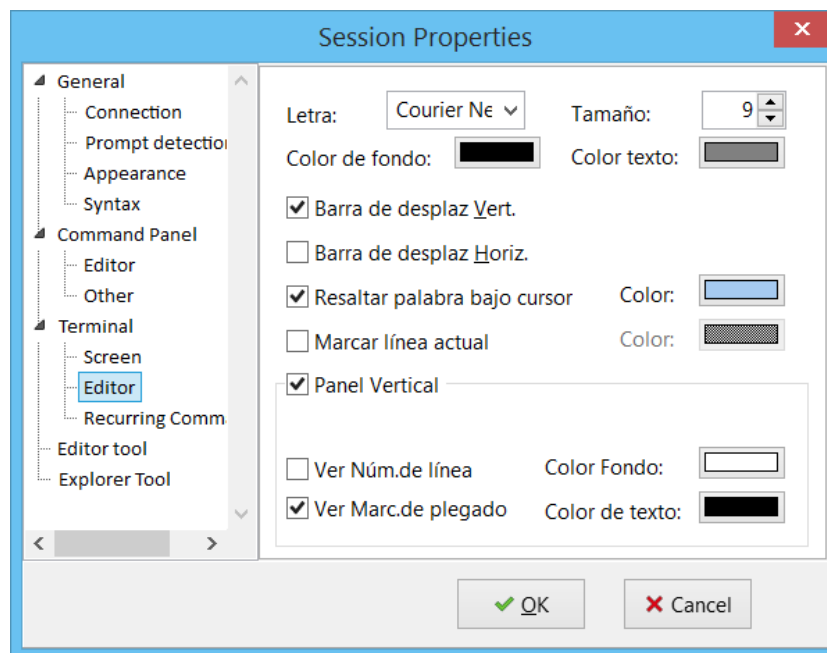
3.5 El Terminal

El terminal tiene la característica de coloreado de texto, en la pantalla del terminal. No se ha implementado el reconocimiento de las secuencias de color que suelen usar algunos “shell” como el ksh o bash. El coloreado lo hace el mismo programa, en base al reconocimiento de ciertas palabras y caracteres claves.

El lenguaje que se usa para el coloreado es el Shell de UNIX. Las palabras reservadas de este lenguaje, suelen ser las mismas que los comandos que se usan comúnmente en el trabajo diario con el terminal, como “echo”, “ls”.

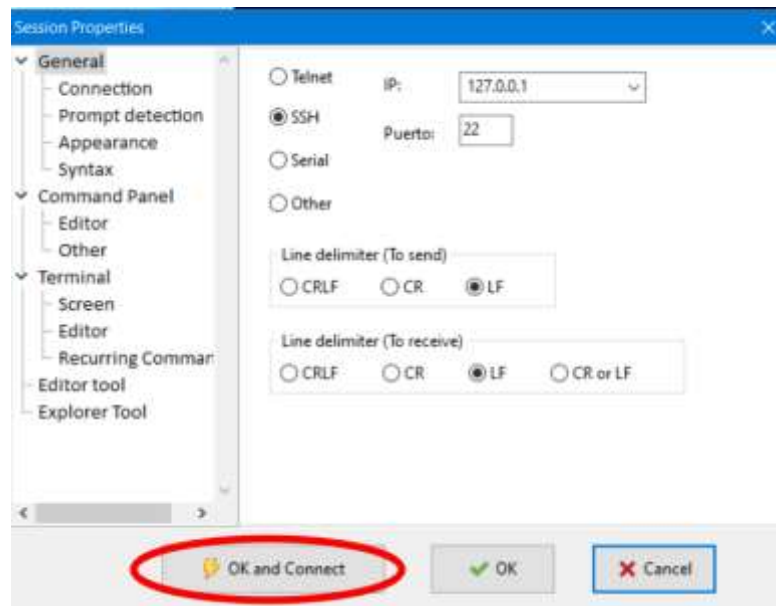
Se colorean los comentarios, palabras reservadas, constantes cadena y el *prompt*, cuando está configurado.

La pantalla del terminal más que una consola VT100, es un editor moderno con resaltado de sintaxis, plegado de código, numeración de línea y resaltado del *prompt* y la línea actual. Además, puede resaltar palabras similares con solo marcar una palabra cualquiera. Para cambiar las configuraciones colores usar la opción “Terminal>Editor”, desde la ventana de propiedades de la sesión:

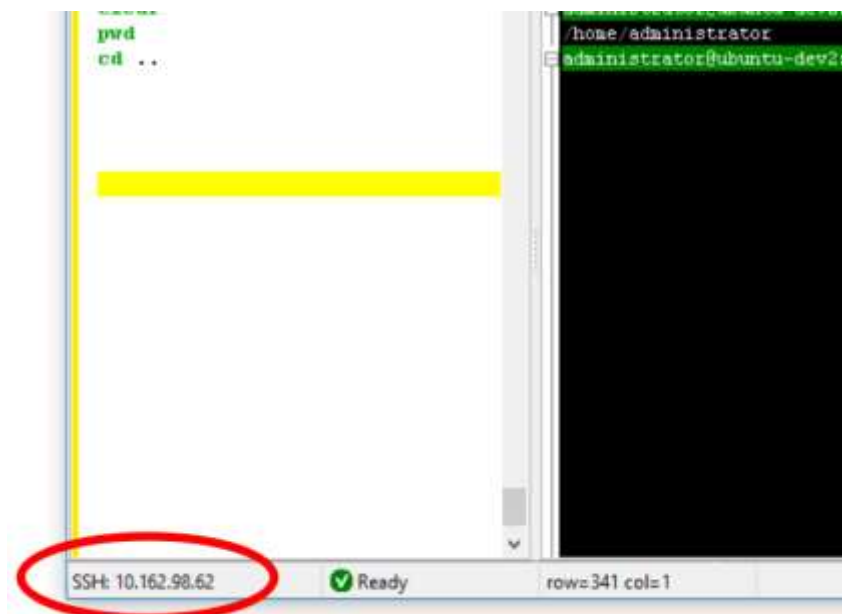


CONECTANDO Y DESCONECTANDO UNA SESIÓN

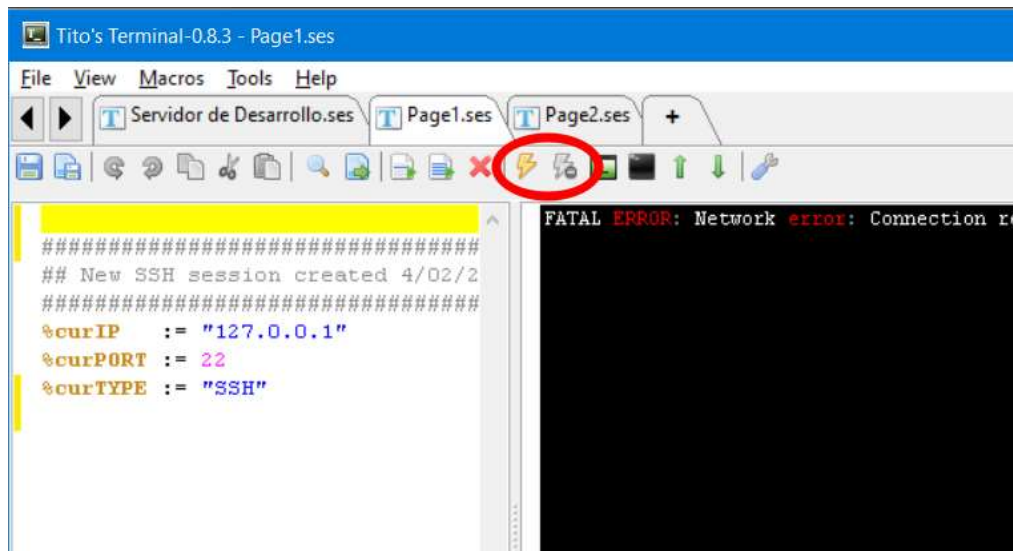
Normalmente, cuando se crea una nueva sesión, se realiza la conexión de forma automática, si se pulsa el primer botón de la configuración de sesión:



Cuando una sesión está conectada se dice que está activa. Los datos de la conexión se pueden apreciar en la barra de estado:



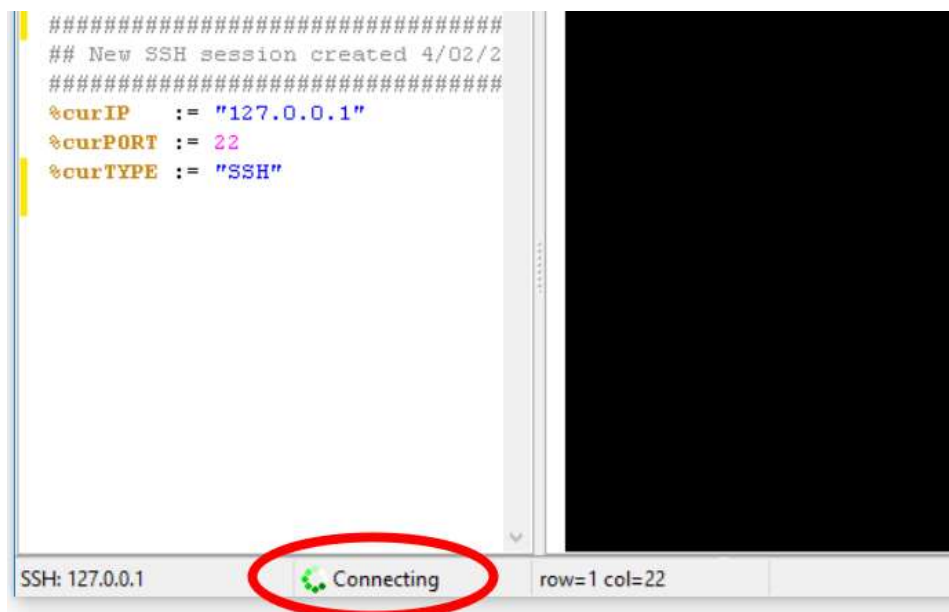
El estado de una sesión se puede apreciar por el estado de los botones de conexión de la barra de herramientas:



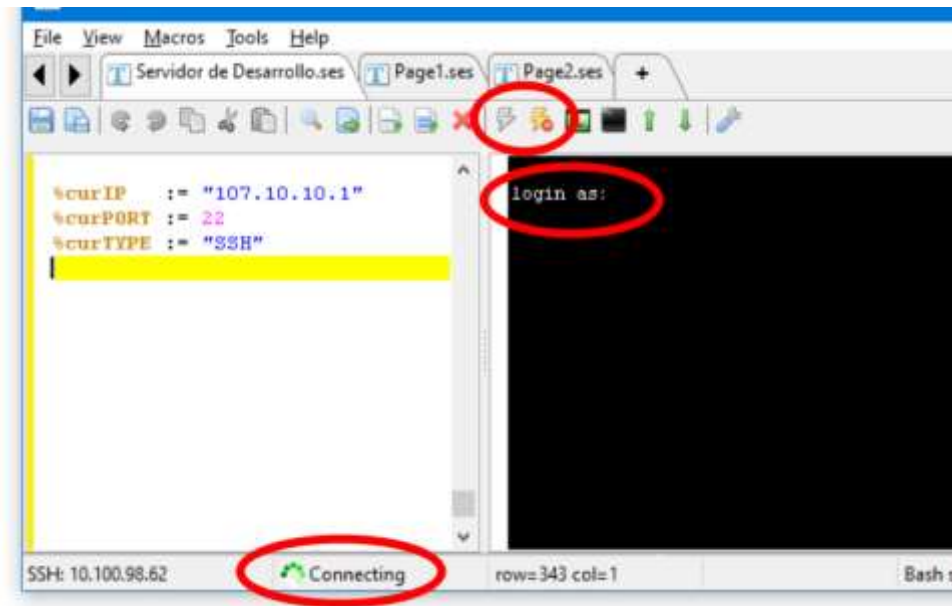
Estos botones también sirven para conectar y desconectar una sesión.

Al iniciar la conexión, con los botones de la barra de herramientas, se usarán las propiedades indicadas en la configuración de la sesión (Tipo de conexión, IP, puerto).

Al conectar una sesión, debe aparecer el ícono de estado de la conexión en la barra de estado:



Cuando la sesión logra conectarse a un servidor o PC, debería aparecer un texto en el Terminal:



Otra forma de iniciar una conexión es ejecutando alguna macro que inicie la conexión (Ver Sección 5).

Otra forma de iniciar sesión es enviando comandos-macro desde el panel de comandos, como se indica en la sección 3.4.

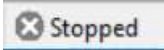
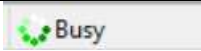
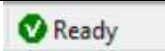
ESTADOS DE LA CONEXIÓN

El estado de una conexión aparece en la barra de estado, en la parte inferior de la ventana:



Una conexión puede encontrarse en distintos estados:

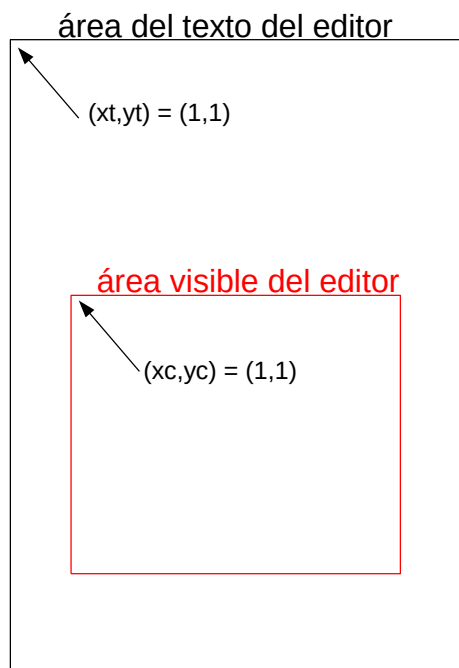
ESTADO	ÍCONO Y ESTADO	DESCRIPCIÓN
--------	----------------	-------------

Conexión detenida		La conexión no se ha iniciado o se ha detenido.
Conexión en proceso		Se está tratando de iniciar una coenxión con el proceso elegido (normalmente Telnet o SSH).
Conexión ocupada		Se ha logrado la conexión pero no aparece el prompt o no se ha configurado la detección del prompt.
Conexión lista		Se tiene una conexión activa y se ha detectado el pormpt.

3.6 Pantalla del Terminal

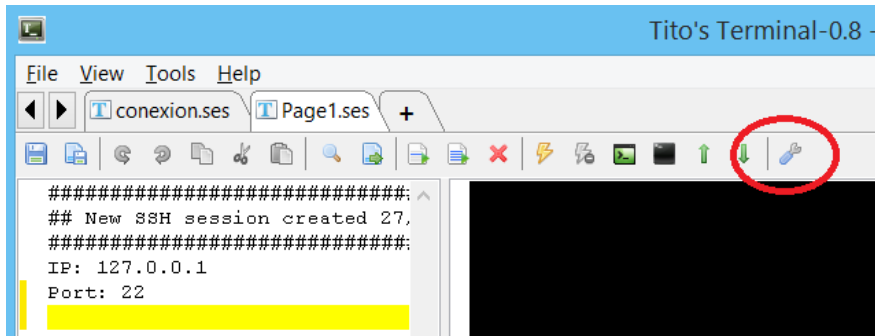
A diferencia de la mayoría de terminales para Telnet o SSH, Tito's Terminal maneja de forma separada el Área Total del Texto, del Área del Terminal, del Área de la ventana visible.

La ventana del terminal es, en realidad, un visor de texto con funciones de terminal. El Área de texto del visor es por lo general mayor que el área de la ventana visible.

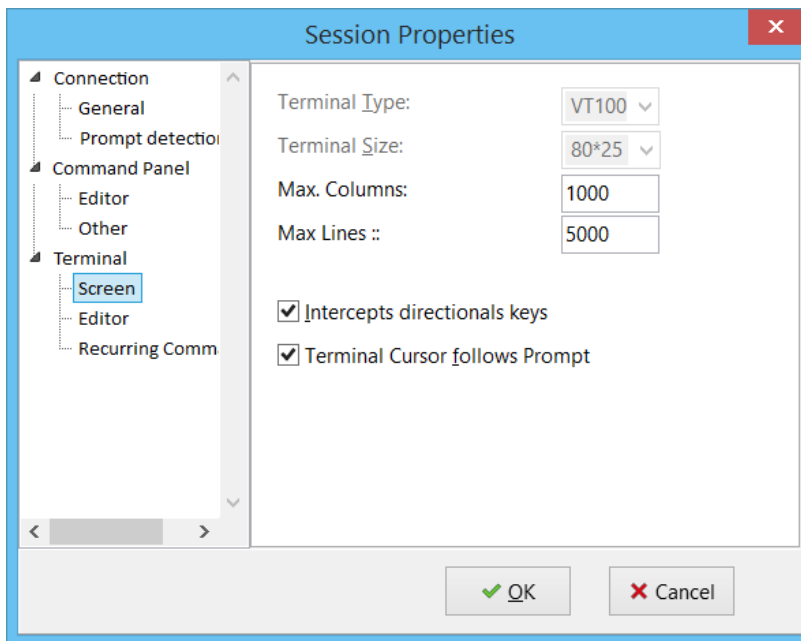


El Área visible depende del tamaño de la ventana principal de la aplicación. Se puede dimensionar en cualquier momento. No depende del tamaño del Área de texto total.

El área del texto total crece conforme van llegando caracteres por el terminal. Inicialmente es pequeña, pero va alcanzando varias líneas, hasta un máximo definido. A partir de allí, se van truncando las líneas superiores. El máximo de líneas que soporta se define en el campo "Líneas Guardadas", en la ventana del menú "Propiedades" de la sesión:



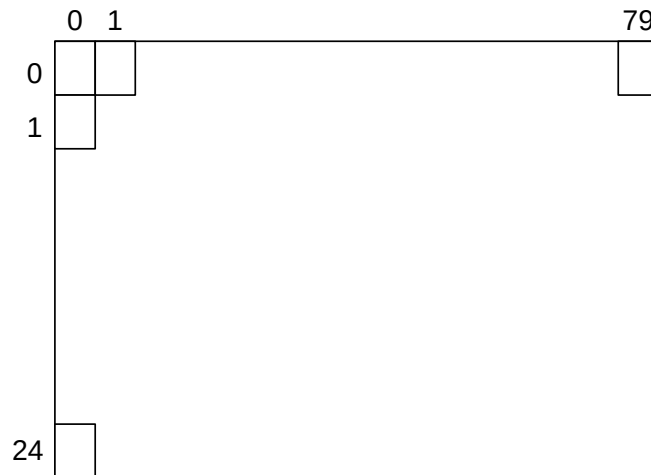
La opción que controla la cantidad de líneas a almacenar es "Terminal>Screen", del árbol de opciones de la izquierda.



También se puede definir aquí, el máximo de columnas que se soportará en el área del texto del editor.

3.6.1 Coordenadas de un Terminal VT100 DE 80 * 25

Un terminal VT100 está diseñado a modo de pantalla única que está orientada a mostrar toda la información en un solo "pantallaza". No está orientado a realizar desplazamientos.



Las coordenadas en un VT100 empiezan en la posición 0,0 y están siempre referidas a la ventana visible del terminal. Es decir que el punto 0,0 será siempre el extremo superior izquierdo y el punto 79, 24 será siempre el extremo inferior derecho, aún después de los desplazamientos de pantalla.

Los límites de las coordenadas del terminal son fijos. No se cambian escribiendo texto ni redimensionando la ventana. Para cambiar el tamaño de una pantalla VT100 se debe negociar con el servidor un nuevo tamaño del terminal.

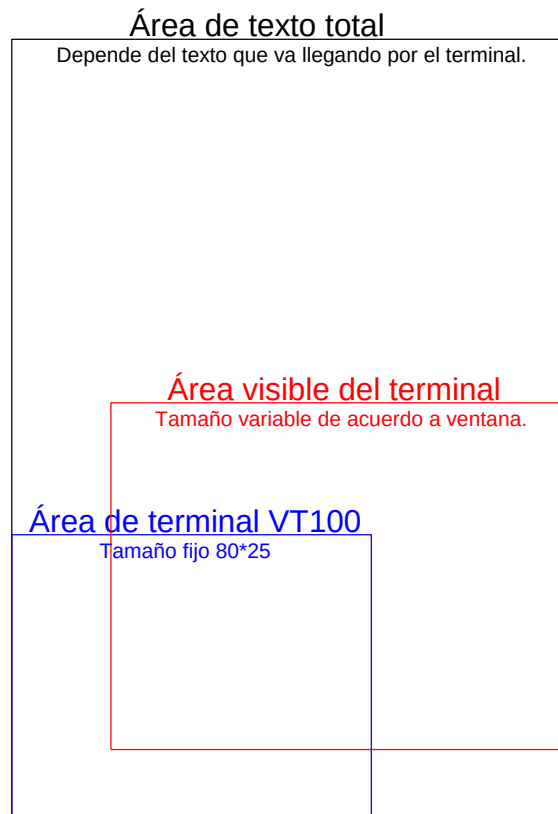
Este tamaño de pantalla es la única que manejan terminales simples como el “telnet” del DOS.

3.6.2 Relación entre las 3 áreas

En el Tito's Terminal existe un área especial para emular a un terminal VT100. Esta área es de tamaño fijo de 80 * 25 caracteres y se ubica siempre al final del área del texto total.

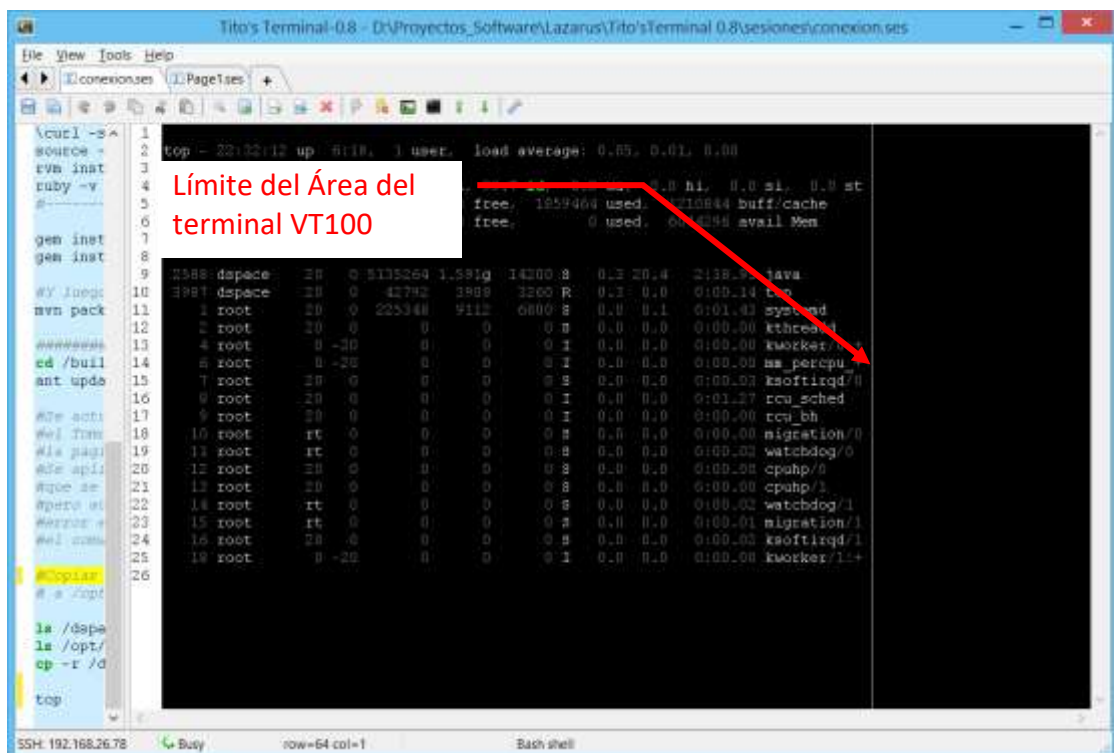
Para que el editor pueda manejar las coordenadas de un terminal VT100, se ha definido un nuevo sistema de coordenadas y además un nuevo cursor. Este nuevo sistema de coordenadas determina un área de trabajo que estará siempre pegada a la parte izquierda del área de las coordenadas de texto.

Una referencia a las 3 áreas se muestra en el gráfico:



La pantalla del VT100 siempre estará alineada al lado izquierdo del Área de texto total.

Tito's Terminal el límite lateral derecho de la ventana del terminal VT100 mediante una línea vertical en la pantalla:



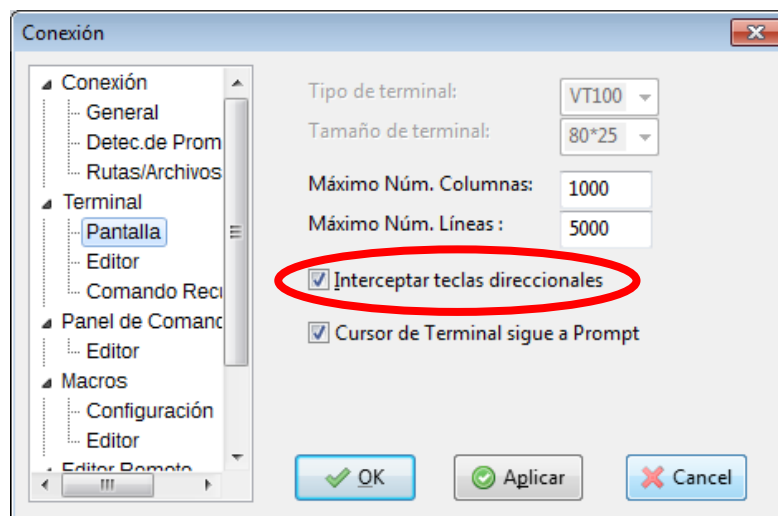
3.6.3 Manejo del Cursor

Como se indicó anteriormente la aplicación maneja separadamente el Área total del texto y el Área correspondiente al terminal VT100.

De la misma forma, se maneja de forma separada, dos cursores. Uno corresponde al del terminal VT100, y lo controla la sesión telnet. Por lo general se encuentra al final de todo el texto. Sirve para editar los comandos que se introducen en la sesión y no se puede variar libremente su ubicación.

De manera separada se trata otro cursor que permite desplazarse libremente por todo el Área total del texto. Este cursor corresponde al visor del texto y su posición, determina la ubicación del Área visible del terminal.

Estos dos cursores suelen coincidir y mantenerse en la misma posición, pero se les puede “desenganchar” en cualquier momento, usando “mouse” para fijar la nueva posición del cursor. También se le puede separar, usando simplemente las teclas direccionales “arriba” o “abajo”, cuando está activada la opción “Interceptar teclas direccionales”, en el menú “Herramientas>Configuración>Pantalla”



En este modo, sin embargo, teclas direccionales “derecha”, “izquierda”, “arriba” o “abajo” ya no serán recibidas por el terminal, sino que servirán para desplazarse libremente por la pantalla de terminal.

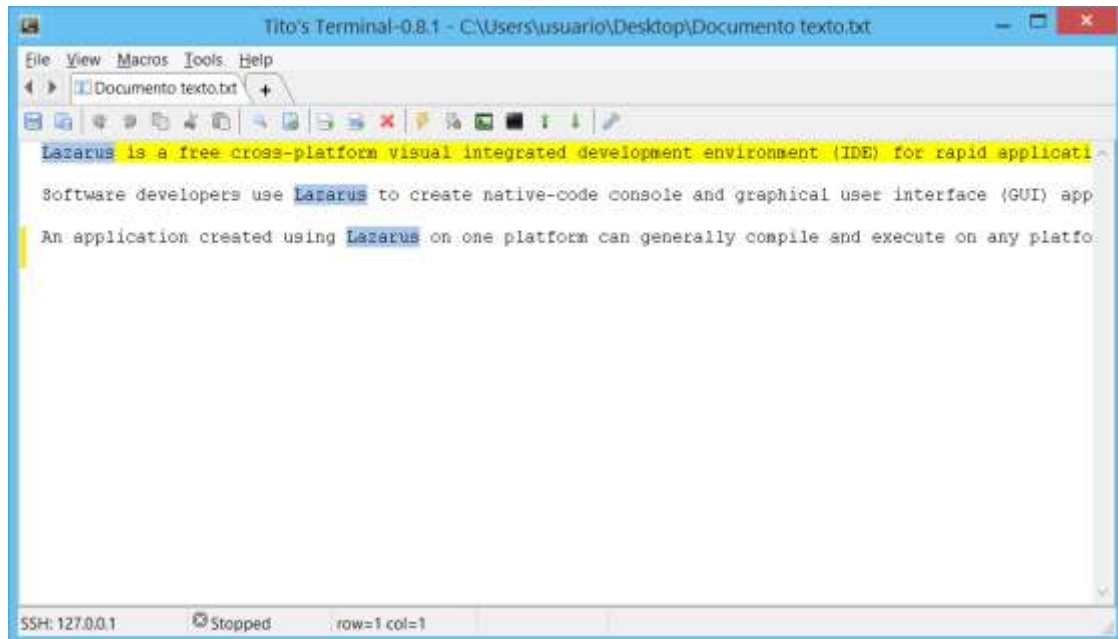
Los cursores coincidirán nuevamente, en cuanto llegue información del terminal, posicionándose en el Prompt. Para evitar que el cursor siga al Prompt, cuando lleguen datos, se debe desactivar la opción “Cursor de Terminal sigue a Prompt”.

Así el cursor no se moverá, aunque lleguen datos por el terminal. Sin embargo, tampoco se realizará el desplazamiento de pantalla.

3.7

3.8 Archivos de texto

Además de las sesiones, Tito's Terminal puede abrir también archivos de texto.



Cuando se abren archivos de texto, se oculta la ventana del terminal, de modo que la interfaz se parece más a un editor de texto común, manteniendo las características de edición que tiene el panel de comandos, como resaltado de sintaxis, de palabras, edición síncrona, y edición multilínea.

Una característica que tal vez se extrañe en este editor es que carece de plegado de líneas largas. Esto significa que, para ver una línea larga completa, se tiene que hacer un desplazamiento horizontal.

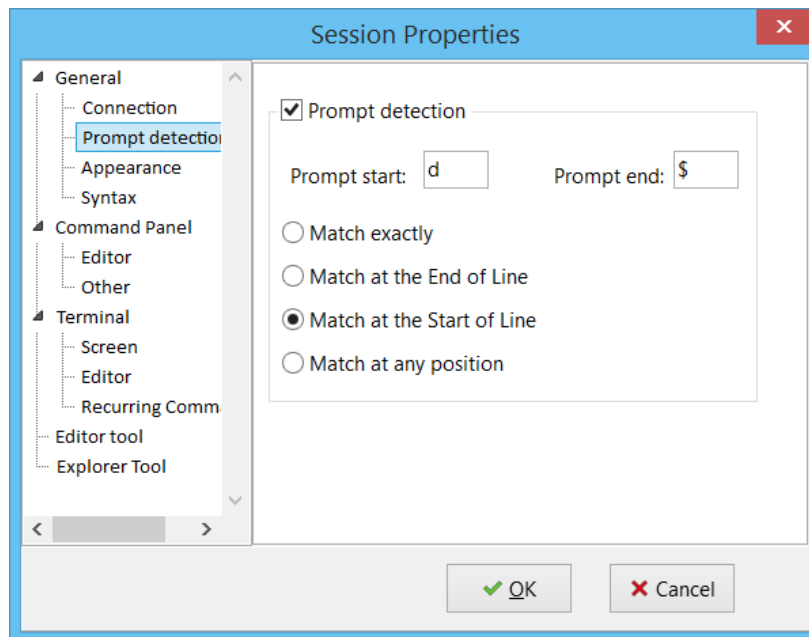
Los archivos de texto se pueden editar y grabar como haría cualquier editor. Los comandos que interactúan con el terminal, dejan de funcionar.

Para que un archivo sea considerado de texto, debe tener extensión *.txt.

4 HERRAMIENTAS

4.1 Detección del Prompt

El coloreado del *prompt*, no se hace de forma directa, sino que se debe primero configurar el reconocimiento del *prompt* en el terminal. Esto se hace desde la opción “General>Prompt detection” desde la ventana de propiedades de la sesión:



La detección del *prompt* se basa en que es posible identificar completamente al *Prompt* en una línea, definiendo los caracteres iniciales y finales que delimitan al *Prompt*.

En la figura mostrada, se han definido los caracteres “[” y “]\$” para el *prompt*. De esta forma se podrán reconocer los siguientes *Prompt*:

```
[user@host]$  
[usuario@localhost ~]$  
[CMD]$
```

Estos Prompt se reconocen bien porque siempre tienen los mismos caracteres iniciales y finales. Pero si el Prompt cambia los caracteres iniciales, o finales, no será fácil detectarlo.

Por ejemplo el prompt del DOS:

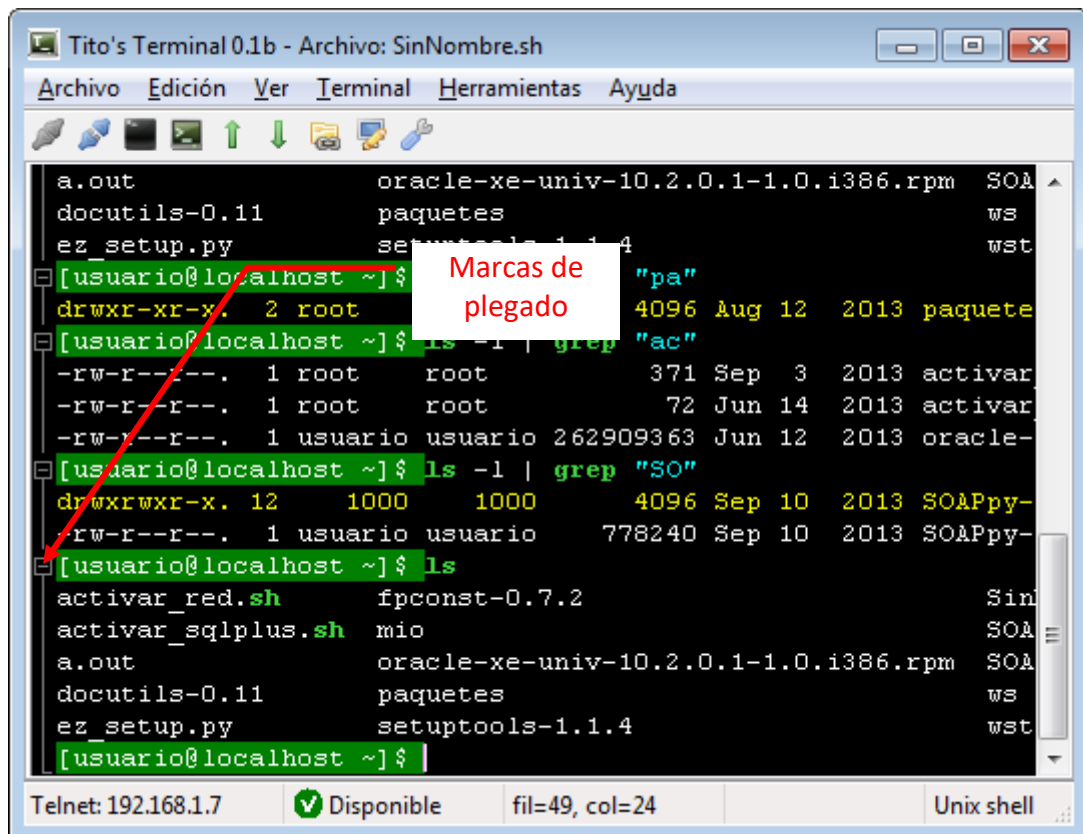
```
C:\SHELL\Usuario>
```

Puede cambiar, el carácter inicial: En este caso, se puede configurar de nuevo el *prompt*, cuando este cambie.

Si no se puede detectar correctamente el *Prompt*, se puede intentar configurar otro *Prompt* en el proceso que esté corriendo en la consola (Shell). O elegir otro grupo de caracteres en la ventana de configuración.

Si el *Prompt* es fijo y no va a cambiar, basta con escribir el Prompt completo en el campo "Inicio Prompt" y dejar el campo "Fin Prompt" en blanco.

Cuando se detecta el Prompt, se aplica un color diferente para ubicarlo fácilmente en el terminal. Además es posible ocultar el texto entre 2 Prompt, porque se genera una marca de plegado de código en el panel lateral (Si es que se ha configurado para que sea visible):



The screenshot shows a terminal window titled "Tito's Terminal 0.1b - Archivo: SinNombre.sh". The window has a menu bar with "Archivo", "Edición", "Ver", "Terminal", "Herramientas", and "Ayuda". Below the menu is a toolbar with various icons. The terminal content shows a list of files and directories, including "a.out", "docutils-0.11", "ez_setup.py", "oracle-xe-univ-10.2.0.1-1.0.i386.rpm", "paquetes", "setuptools-1.1.4", "SOA", "ws", and "wst". The prompt "[usuario@localhost ~]\$" is highlighted in green. A red arrow points to this prompt, and a white box with red text "Marcas de plegado" points to the green highlight. The terminal also shows the output of the command "ls -l | grep 'ac'", which lists files like "activar_red.sh", "activar_sqlplus.sh", "a.out", "docutils-0.11", "ez_setup.py", and "setuptools-1.1.4". The status bar at the bottom shows "Telnet: 192.168.1.7", "Disponibile", "fil=49, col=24", and "Unix shell".

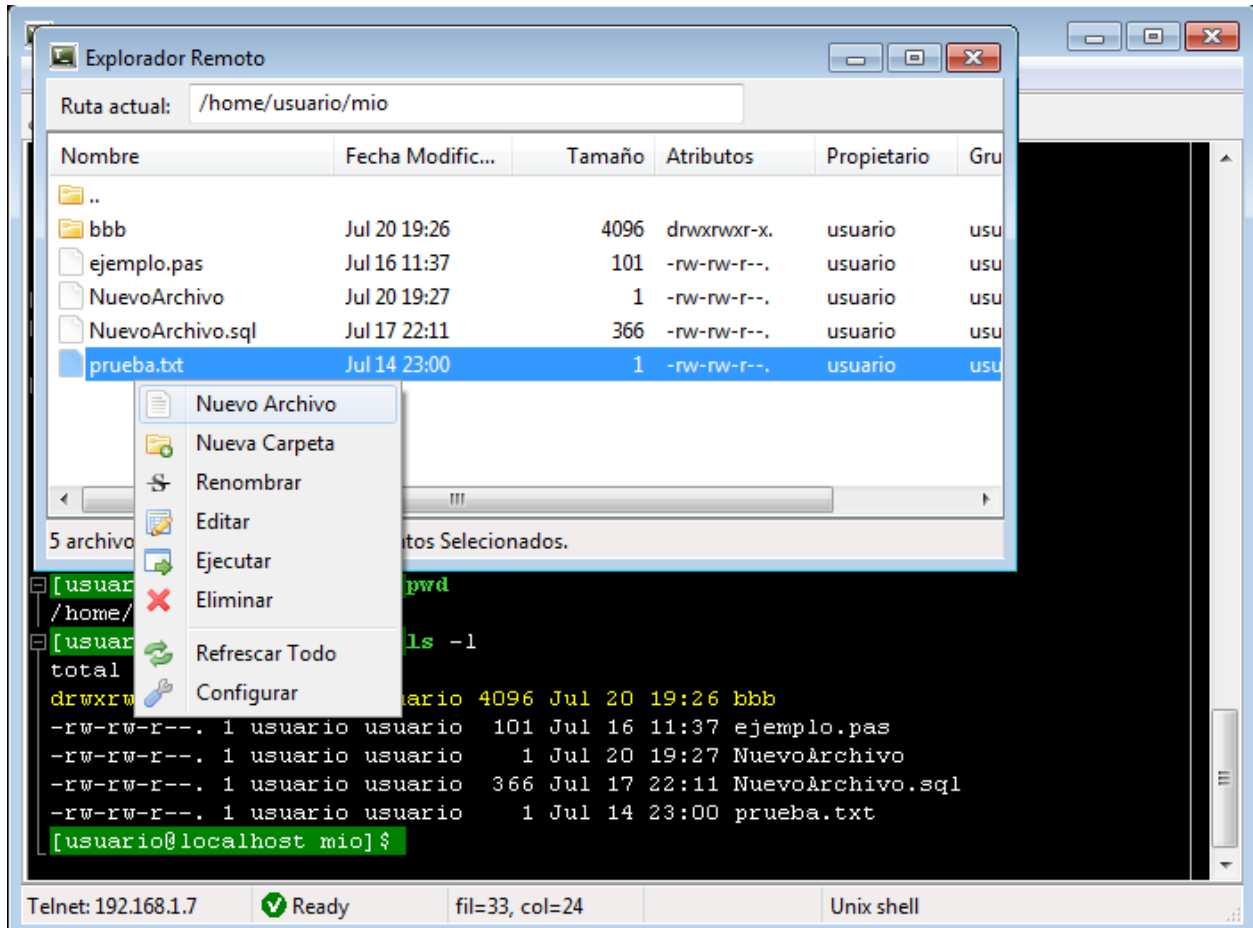
Para configurar automáticamente, la detección del Prompt en Tito's Terminal, se debe usar el menú "Terminal>Detectar Prompt". También se puede acceder desde un botón la barra de herramientas del Terminal.

El método consiste en detectar automáticamente los caracteres iniciales y finales. Este método no es seguro, pero funcionará bien en la mayoría de configuraciones del Prompt.

Tener bien configurado el reconocimiento del Prompt, es importante no solo por fines de coloreado, sino que también ayuda para el trabajo del Editor Remoto y del Explorador Remoto (Ver más adelante).

4.2 Explorador Remoto

El explorador remoto es un accesorio que permite usar una conexión Telnet o SSH, para mostrar un explorador de archivos visual, de forma similar a como se hace en Windows.

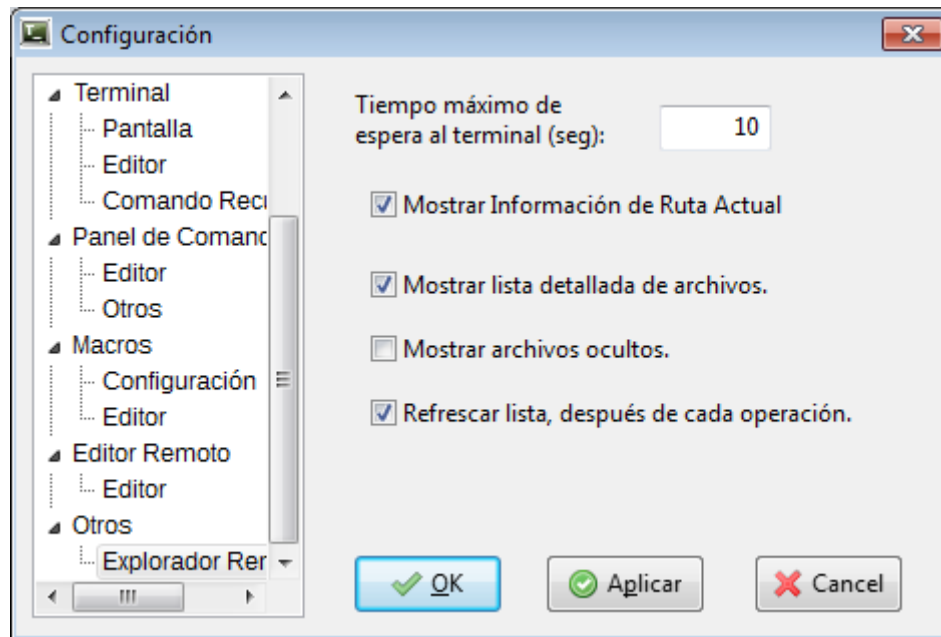


En el explorador remoto se puede navegar a través de los directorios y crearlos o eliminarlos. También se puede crear, eliminar, y renombrar archivos.

Para que funcione correctamente el explorador remoto, es necesario que esté activada y configurada la opción “Reconocimiento de Prompt”.

Si el sistema no puede reconocer el “Prompt”, no se podrán usar las opciones de exploración remota. Además de acuerdo al tipo de shell y sistema operativo, podría no obtenerse un funcionamiento adecuado. Esta opción solo está prevista para trabajar en conexiones Telnet o SSH.

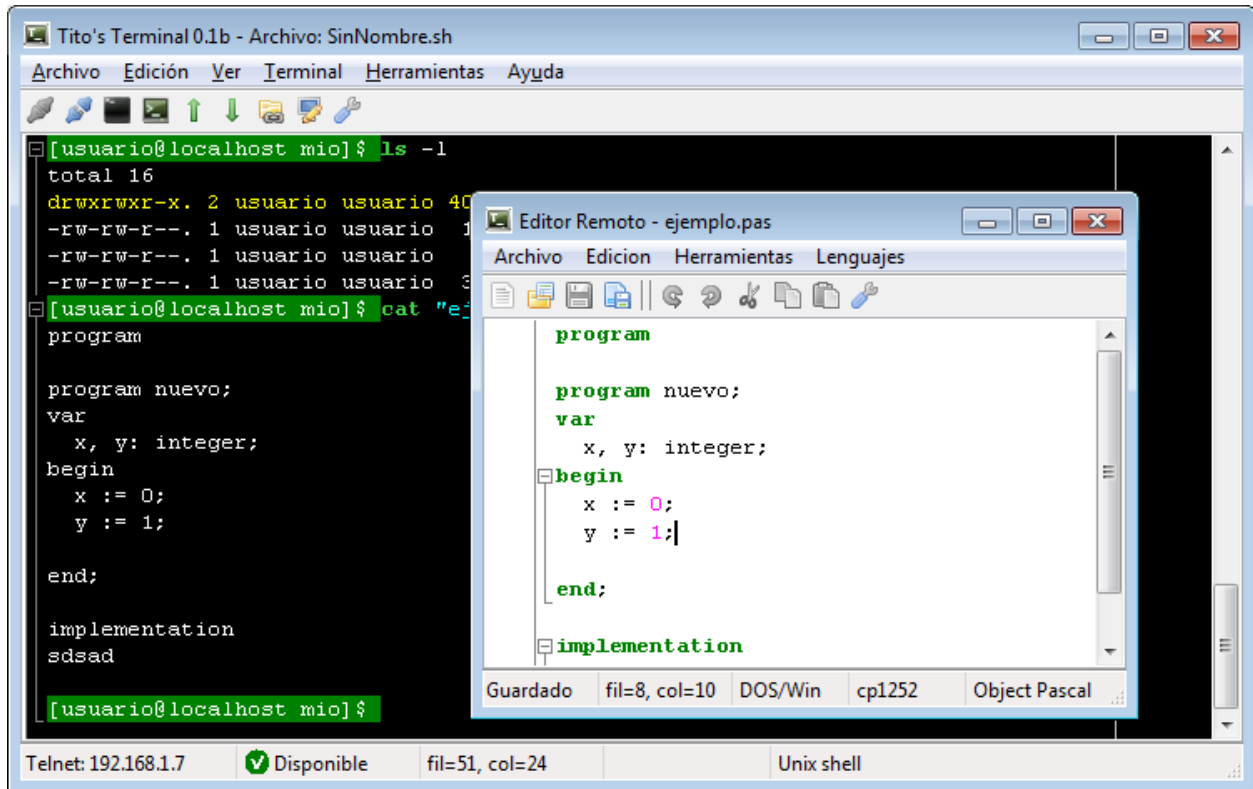
Para conexiones lentas, puede haber un retraso en refrescar la información del explorador remoto. Para estos casos se recomienda desactivar la opción “Mostrar Lista detallada de archivos”, en las opciones de configuración del explorador remoto.



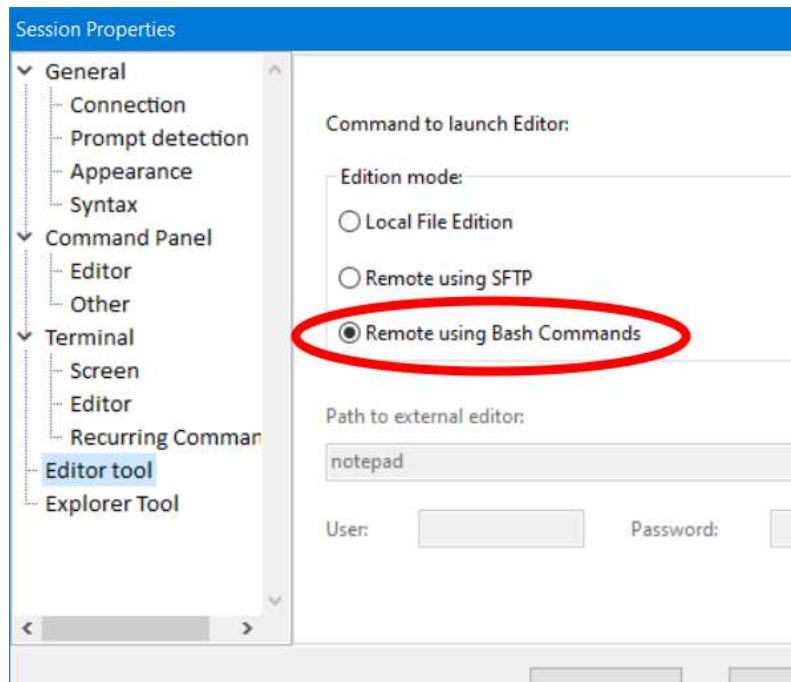
El editor remoto se puede abrir de forma abreviada usando la combinación de teclas <Ctrl>+E.

4.3 Editor Remoto

El editor remoto es una funcionalidad que permite editar en una ventana especial, un archivo de texto del servidor remoto. La edición se hace en el “Editor remoto” del programa y se comunica con el servidor mediante comandos “bash” de forma transparente para el usuario.



Para que estén activas las opciones de edición remota, debe estar configurado, en las propiedades de la sesión, que el editor se use en forma remota:



También es necesario que esté activada y configurada la opción “Reconocimiento de Prompt” y que se haya detectado el “Prompt” de la conexión SSH o Telnet.

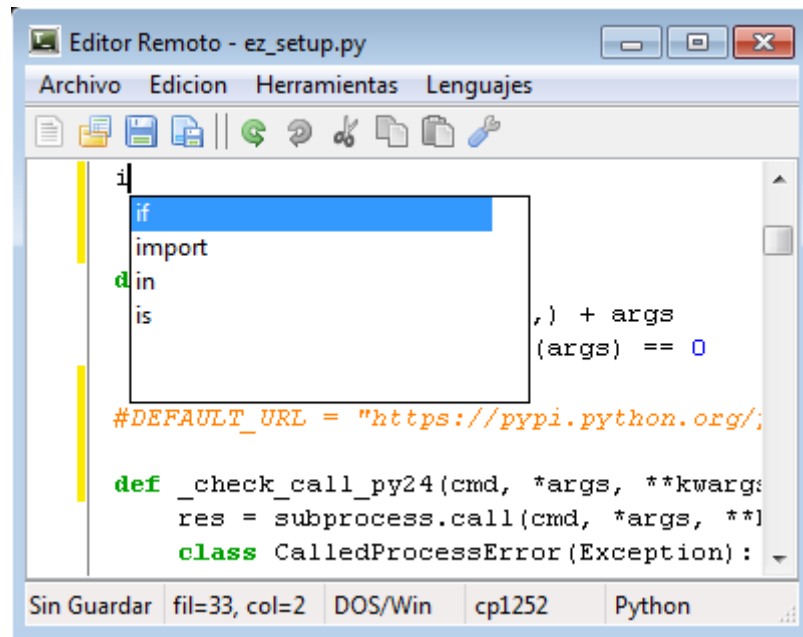
Si el sistema no puede reconocer el “Prompt”, no se podrán usar las opciones de edición remota. Además, de acuerdo al tipo de shell y sistema operativo, podría no lograrse la edición remota de archivos. Esta opción solo está prevista para trabajar en conexiones Telnet o SSH.

Como las capacidades para mover información al servidor remoto mediante Telnet/SSH, son limitadas, no es factible para la edición de archivos grandes. Solo se recomienda el uso para archivo de 200 líneas o menos, pero dependerá también de la velocidad de la red.

El editor remoto se puede abrir de forma abreviada usándola combinación de teclas <Ctrl>+<F2>.

Una vez abierto el editor remoto, es posible usar las opciones “Abrir”, “Guardar” y “Guardar como...”, del menú del editor remoto.

El editor remoto tiene incluido la opción de coloreado de sintaxis, plegado de código y autocompletado, para diversos lenguajes, de forma similar al Panel de Comandos.



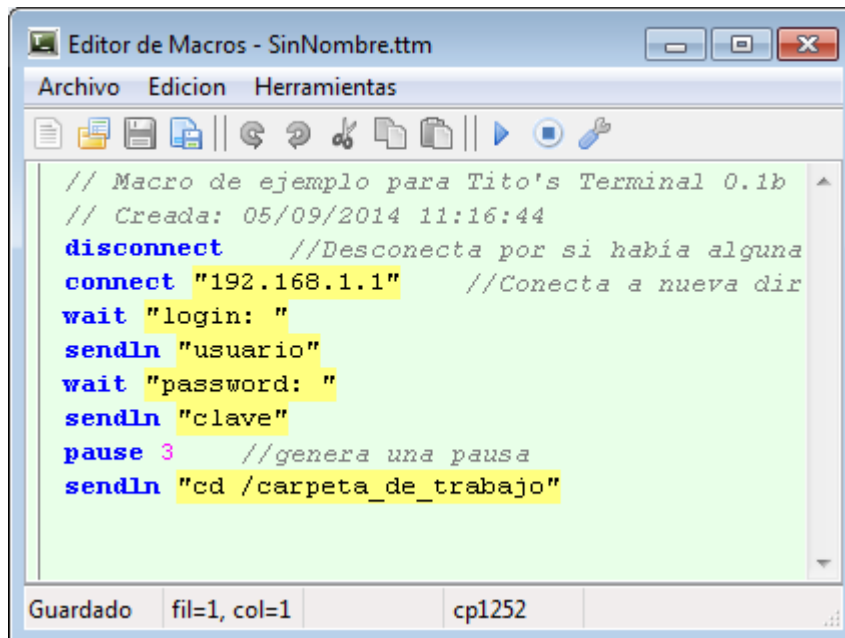
The image shows a screenshot of a remote editor window titled "Editor Remoto - ez_setup.py". The window has a menu bar with "Archivo", "Edicion", "Herramientas", and "Lenguajes". Below the menu is a toolbar with icons for file operations and editing. The main text area displays Python code. A blue selection box highlights the word "if" on the first line of a conditional block. The code includes an "import" statement, a "def" function definition for "_check_call_py24", and a "class" definition for "CalledProcessError". The status bar at the bottom shows "Sin Guardar", "fil=33, col=2", "DOS/Win", "cp1252", and "Python".

```
if
import
def _check_call_py24(cmd, *args, **kwargs):
    res = subprocess.call(cmd, *args, **kwargs)
    class CalledProcessError(Exception):
        pass
```

Sin Guardar fil=33, col=2 DOS/Win cp1252 Python

4.4 Editor de macros

El Editor de macros permite crear, modificar, o probar las macros, en un pequeño entorno de Edición. Para mostrar el editor de macros, se debe usar el menú “Ver>Editor de Macros”



Por defecto los archivos de macros se guardan en la carpeta “/macros”, en la carpeta principal del programa.

También se pueden ejecutar directamente las macros desde el menú “>Herramientas>Ejecutar Macro”, seleccionando el nombre de la macro.

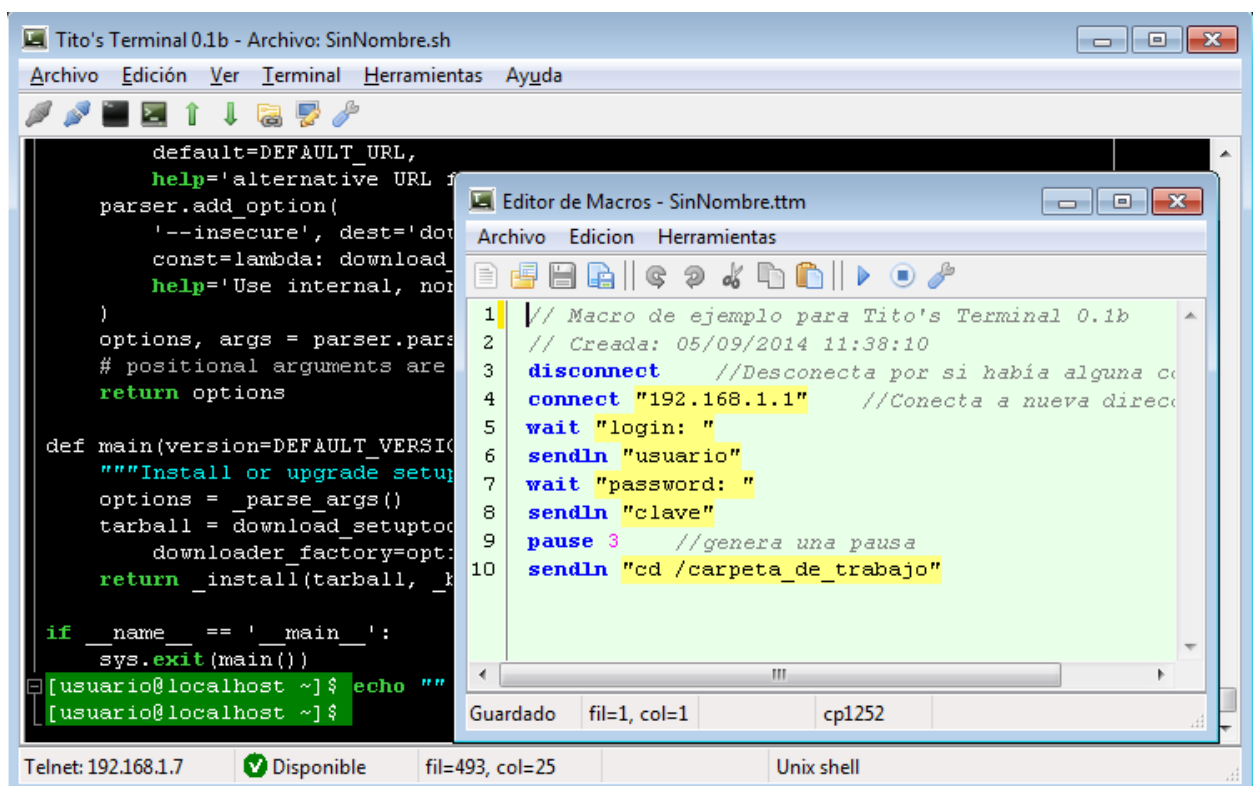
Para más información sobre la creación de Macros, ver la sección de automatización.

5 AUTOMATIZACIÓN

El terminal soporta la ejecución de macros programados en un sencillo lenguaje de programación de alto nivel.

El lenguaje de programación está basado en el lenguaje del terminal TeraTerm con algunas diferencias.

Las macros se pueden crear en un editor normal de texto o se pueden editar con el editor de macros integrado que permite la ejecución directa, sin necesidad de salir del editor. El editor de macros además incluye resaltado de sintaxis y ayuda contextual.



Desde el panel de macros se pueden crear, modificar y ejecutar las macros. Las macros se guardan por defecto en la carpeta `"/macros"` en la carpeta principal del programa.

Las macros también se pueden ejecutar directamente desde el menú "Macros", seleccionando el nombre de la macro. Las macros en la carpeta `"/macros"` se cargan en este menú de forma automática al iniciar el programa.

5.1 Variables y Tipos de datos

Las variables son elementos del lenguaje que permiten almacenar valores de tipo numérico o de cadena.

No es necesario declarar las variables antes de usarlas dentro del programa.

Los nombres de las variables pueden tener solo caracteres alfanuméricos pero no pueden empezar con un dígito numérico.

El lenguaje solo diferencia tres tipos de datos:

1. Números (solo enteros).
2. Cadenas.
3. Booleanos.

El sistema hará conversiones automáticas de tipo de acuerdo al contexto de las operaciones que se realice.

Las fechas se tratan como número.

Las asignaciones se hacen con el símbolo: ":="

X:=1

Para crear una variable, solo hay que asignarle un valor inicial. Así en la siguiente instrucción, se crean tres variables de diferentes tipos:

```
x := 0 //crea variable entera
cad := "hola" //crea variable de cadena
res := false //crea variable booleana
```

5.2 Estructura del lenguaje

Un programa está dividido en instrucciones. Las instrucciones se separan por saltos de línea.

Cada línea podrá contener una sola instrucción.

No se usa ";" como delimitador de instrucciones, en la forma en que es usado en otros lenguajes. Si se pusiera ";", se obtendrá un mensaje de error.

Los comentarios de una línea empiezan con los caracteres "//". Los de varias líneas empiezan, están delimitados por los caracteres: /* ... */

Todas las instrucciones se consideran también como expresiones.

Un programa no tiene que seguir una secuencia especial (bloques “Main” o encabezados). Las instrucciones se pueden escribir desde el principio. El siguiente es un ejemplo sencillo:

```
//Programa ejemplo
IP := "192.168.1.3"
MESSAGEBOX "Vamos a intentar conectarnos a: " + ip
CONNECT IP
```

5.3 Referencia a Instrucciones

CONNECT

Abre una sesión Telnet a una dirección específica.

DISCONNECT

Termina la sesión Telnet que está en curso.

SENDLN

Envía una cadena al terminal agregando un salto de línea al final.

WAIT

Espera a que aparezca una cadena específica en el terminal.

PAUSE

Sintaxis: PAUSE <tiempo en segundos>

Genera un retardo de tiempo antes de continuar con la ejecución del programa. El tiempo se da en segundos.

STOP

Termina la ejecución de la macro actual.

END

Termina el aplicativo. Cierra el programa.

LOGOPEN

Sintaxis: LOGOPEN <nombre de archivo>

Abre un archivo de texto para usarlo como registro de la salida del terminal.

LOGWRITE

Escribe una cadena en el archivo de registro.

LOGCLOSE

Cierra el archivo de registro actual.

LOGPAUSE

Pausa la escritura de datos en el archivo de registro.

LOGSTART

Reinicia la escritura de datos en el archivo de registro.

FILEOPEN

Abre un archivo para leer o escribir datos.

FILECLOSE

Cierra un archivo que estaba abierto previamente.

FILEWRITE

Escribe datos en un archivo abierto previamente con FILEOPEN.

MESSAGEBOX

Muestra un mensaje en pantalla en una pequeña ventana modal. La ejecución del programa se detiene hasta que el usuario pulse "Aceptar" en el mensaje de texto.

CAPTURE

Inicia la captura de la salida del terminal en una variable cualquiera.

ENDCAPTURE

Finaliza la captura iniciada con CAPTURE

EDIT

Lanza el editor remoto para editar algún archivo del servidor.

DETECT_PROMPT

Configura la detección de prompt con datos de la línea actual, para que la interprete como si fuera el prompt.

NOTA: Algunas instrucciones no están disponibles aún en la presente versión.

5.4 Expresiones

Las expresiones están compuestas de operandos y operadores.

Los operadores reconocidos por orden de jerarquía son:

JERARQUÍA	OPERADOR
1	">>", "<<", ">+", ">-"
2	"="
3	"&&", " ", "!", " !"
4	"==", "<>", ">", ">=", "<", "<=", "~"
5	"+", "-", " ", "&"
6	"*", "/", "\", "%"
7	"<", ">"
8	"^", "++", "--", "+=", "-=", "*=", "/="

5.5 Variables Predefinidas

Existe un grupo de variables que existen siempre y que sirven para configurar los parámetros de la conexión. Estas son:

Timeout	Es el tiempo de espera máximo para las instrucciones que esperan respuesta de la conexión. Se indica en segundos.
curIP	Es la dirección IP de la conexión actual.
curTYPE	Es el tipo de conexión de la conexión actual. Los posibles valores son: "Telnet", "SSH", "Serial" y "Other".
curPORT	Es el número de puerto, de la conexión actual: solo es válido para conexiones Telnet/SSH.
curENDLINE	Es el tipo de delimitador que se envía como salto de línea a la sesión.
curAPP	Es el aplicativo que se usa para la conexión, cuando se selecciona el tipo de conexión "Other".
promptDETECT	Habilita la detección del Prompt.
promptSTART	Configura la cadena inicial del Prompt, para su detección.
promptEND	Configura la cadena final del Prompt, para su detección.

5.6 Conexiones

Para realizar una conexión Telnet, se puede usar la instrucción CONNECT:

```
CONNECT "192.168.1.1" //Conecta a dirección IP
```

Las conexiones Telnet, se realizan al puerto 23 por defecto.

Para conexiones SSH, se usa la instrucción CONNECTSSH:

```
CONNECTSSH "192.168.1.1" //Conecta a dirección IP
```

Las conexiones SSH, se realizan al puerto 22 por defecto.

Una conexión típica debe incluir además la instrucción DISCONNECT, para cerrar alguna posible conexión anterior:

```
DISCONNECT  
CLEAR  
CONNECT "192.168.1.1"
```

La instrucción CLEAR, es necesaria, cuando se desea limpiar primero la pantalla.

Para especificar más opciones de la conexión, como el puerto, se deben usar las variables predefinidas:

```
DISCONNECT //Desconecta por si había alguna conexión  
CLEAR //limpia la pantalla  
curIP := "192.168.1.1"  
curPORT := 23 //configura el puerto  
curTYPE := "Telnet"  
curENDLINE := "LF" //El tipo de salto de línea a enviar  
CONNECT //Inicia conexión
```

5.7 Ejecución desde el Panel de Comandos

Es posible ejecutar instrucciones, del lenguaje de automatización, directamente desde el Panel de Control, como se indica en la Sección 3.3.